CS396 Project Phase 2 Report

epaData: Analysis, Impact Assessment, and Prediction for EPA Power-Sector Emissions Data

Raaj Banga Arnav Bawankule

November 2026

Abstract

Phase 1 of epaData built a database of the annual operating and emissions data that U.S. power-plant units report to the EPA Clean Air Markets Program Data service (CAMPD), with retrieval, upload, validation, search, and export. Phase 2 adds analysis on top of that database. An *Analyze* page selects unit-years with the Phase 1 filters and then reports descriptive statistics, derived indicators (emission intensities, heat rate, capacity factor, year-over-year change), interactive charts, group comparisons with confidence intervals and hypothesis tests, and EPA TRACI 2.2 potential-impact indicators for CO_2, SO_2, and NO_x. A *Predict* page compares a baseline, Ridge regression, and a decision tree that predict a unit's annual NO_x emissions. On the held-out years 2024–2025 the tree reaches $R^2 = 0.951$ (95 % interval 0.937–0.963), against 0.827 for Ridge and 0.489 for the baseline. A simpler rule, last year's NO_x rate times this year's heat input, is more accurate than both models; the report shows why and what the models add. The same models drive one-variable sensitivity curves and named scenarios, with a warning when an input leaves the range of the training data. Every result can be exported with the metadata needed to reproduce it. This report describes the analytical questions, the design, the methods and their assumptions, the results, the testing, and the limitations.

Contents

1 Project description **4**

1.1 Problem 4

1.2 Analytical questions 4

1.3 Intended users 4

1.4 Objectives 4

1.5 Scope 5

2 Functions implemented **5**

2.1 Analytical dataset builder 5

2.2 Derived indicators 7

2.3 Descriptive statistics 7

2.4 Interactive visualization 8

2.5 Comparative analysis 8

2.6 Environmental and health impact indicators 9

2.7 Prediction . . . 9
2.8 Sensitivity and scenario analysis . . . 9
2.9 Saving and exporting results . . . 9
2.10 Requirement traceability . . . 9
2.11 Planned functions not completed or changed . . . 11

3 System design **12**
3.1 Architecture . . . 12
3.2 Life cycle of an analysis . . . 12
3.3 Database additions . . . 12
3.4 Data flow and reproducibility . . . 13

4 Technical details **13**
4.1 Languages . . . 13
4.2 Frameworks and libraries . . . 13
4.3 External services . . . 14
4.4 Scripts and configuration . . . 14

5 Methods **14**
5.1 Variables and missing values . . . 14
5.2 Descriptive statistics and correlation . . . 15
5.3 Comparisons and tests . . . 15
5.4 TRACI impact assessment . . . 15
5.5 Prediction . . . 16

6 Results **17**
6.1 Q1: emissions trends and the shift from coal to gas . . . 17
6.2 Q2: NO_x rate and post-combustion control . . . 18
6.3 Q3: impacts by state . . . 20
6.4 Q4: prediction . . . 20

7 Highlighted features **24**
7.1 One variable catalogue for every analysis . . . 24
7.2 TRACI impacts as ordinary variables . . . 25
7.3 Prediction without Python on the server . . . 26
7.4 Exports that can be reproduced . . . 26

8 Testing and results **27**
8.1 Approach . . . 27
8.2 Unit tests . . . 27

8.3 Representative test cases . . . 28
8.4 Performance . . . 28
8.5 Defects found and corrected . . . 29
8.6 Known issues . . . 29
8.7 Assessment against the goals and objectives . . . 30

9 Discussion **30**
9.1 Effectiveness of techniques and tools . . . 30
9.2 Technical choices and alternatives . . . 30
9.3 Challenges and lessons learned . . . 31

10 Limitations and future improvements **31**
10.1 Current limitations . . . 31
10.2 Future improvements . . . 32

11 Individual contributions **32**

12 References and acknowledgments **33**
12.1 Data . . . 33
12.2 Reused code and assets . . . 33
12.3 Generative AI disclosure . . . 33

1 Project description

1.1 Problem

Phase 1 made CAMPD data easy to find and to trace to its source: a user can list the Kentucky coal units that emitted more than 500,000 tons of CO_2 in 2025 and see which retrieval or upload supplied each value. It could not answer the questions that usually follow. Is the sector's CO_2 falling because plants are cleaner or because generation is moving from coal to gas? Do units with selective catalytic reduction (SCR) emit less NO_x per unit of fuel, and by how much? Which states contribute most to acidification, and does the ranking change once the result is divided by the electricity produced? How much NO_x would a given unit emit if it burned 5 % less fuel or added a control?

Answering these questions needs four things: statistics over many unit-years with a consistent rule for missing values, indicators that are comparable across plants of different size, a documented link from emitted mass to potential impact, and a prediction model that is evaluated on data it has not seen before it is used for what-if analysis.

1.2 Analytical questions

Phase 2 is organized around four questions. Section 6 answers each one with the application.

Q1. How have emissions and emission intensity changed between 2015 and 2025, and how much of the change is a shift from coal to gas?

Q2. How large is the difference in NO_x emission rate between units with and without post-combustion NO_x control, and is it larger than chance would explain?

Q3. Which states contribute most to each TRACI impact category, in total and per MWh?

Q4. How well can a unit's annual NO_x emissions be predicted from its operating data, attributes, controls, and history, and how do the predictions respond to changed inputs?

1.3 Intended users

The specification lists environmental researchers, power-sector analysts, regulators, facility managers, students, and the interested public. Phase 2 assumes these users know the subject but not statistical software. Every statistic is shown with its unit and definition, every test with a plain-language reading, and every model result with a note that it describes the model, not a measured effect.

1.4 Objectives

The course defines twelve objectives for the two-phase project. Phase 1 covered O1–O7, O11, and O12. Three were left for Phase 2:

O8. Calculate environmental indicators.

O9. Score alternatives on several indicators (multi-indicator scoring).

O10. Present evaluation results in interactive charts.

The Phase 2 specification itself sets nine goals: descriptive statistics; comparisons of facilities, units, fuels, states, and periods; interactive visualization; TRACI impact indicators; trends, relationships, and unusual observations; prediction models; sensitivity and scenario analysis; saving

and exporting results; and clear explanation of methods, assumptions, results, and limitations. The specification does not ask for multi-indicator scoring, so O9 was not built (Section 2.11). Phase 2 also completes O11 (test and deploy), which Phase 1 met only partly: the application is now hosted at `https://cs396-project.vercel.app`. Section 8.7 assesses every goal and objective.

1.5 Scope

Phase 2 uses the CAMPD records already in the Phase 1 database and the TRACI 2.2 characterization factors for the three pollutants CAMPD reports (CO_2, SO_2, and NO_x emitted to air). Analyses default to the complete reporting years 2015–2025 retrieved from the CAMPD API. The older uploaded years (1996–1998 and 2006–2008) and the partial year 2026 can still be selected but are excluded from model training. The prediction target is annual NO_x mass per unit-year (Section 5.5).

2 Functions implemented

Two pages were added to the navigation bar: *Analyze* (`/analyze`), with the tabs Overview, Distribution, Relationships, Compare groups, and TRACI impacts, and *Predict* (`/predict`), with the tabs Models and What-if. Each tab answers one question about the selection and opens on a chart; controls specific to a chart sit on the chart, and longer definitions and caveats are under an *About this chart* note. As in Phase 1, the URL holds the full state of a page (filters, tab, and every choice on the tab), so any view can be reloaded, bookmarked, or shared.

The first version organized Analyze by technique: a Dataset tab of definitions, a Statistics tab of numbers, and a Charts tab with seven sub-tabs. At the progress review it read as a data dump that did not look like the rest of the site, so both pages were reorganized by question, with the same filter bar, search, layout, export buttons, and loading and error states as the Explore and Map pages. Every function was kept, and links to the old tabs open the tab that now holds their content. One search box with suggestions now serves every page: typing a name, operator, county, or ID lists matching facilities, and pressing Enter interprets a description ("coal units in Kentucky with high CO2") as filters, as on the Phase 1 Explore page. On the Map page a picked facility is centered and ringed.

2.1 Analytical dataset builder

The Analyze page reuses the Explore page's filter bar: reporting-year range; state, facility, and unit; primary and secondary fuel; unit type; SO_2, NO_x, and particulate controls; emissions program; operating status; record origin; audit flags; and minimum and maximum operating time, gross load, heat input, and emitted mass. The selected unit-years are the analytical dataset for every tab. The Overview tab (Figure 1) shows how many unit-years, facilities, units, and years were selected, then one chosen variable as a trend per year, a state map, and the fifteen highest or lowest facilities or units. A collapsed *Data quality and variables* section lists the fields the selection can be grouped by, with their number of distinct values, and every variable with its kind, unit, definition, how groups combine it, and how many of its values are missing. A *Download all* button exports every selected unit-year with all variables (Section 2.9).

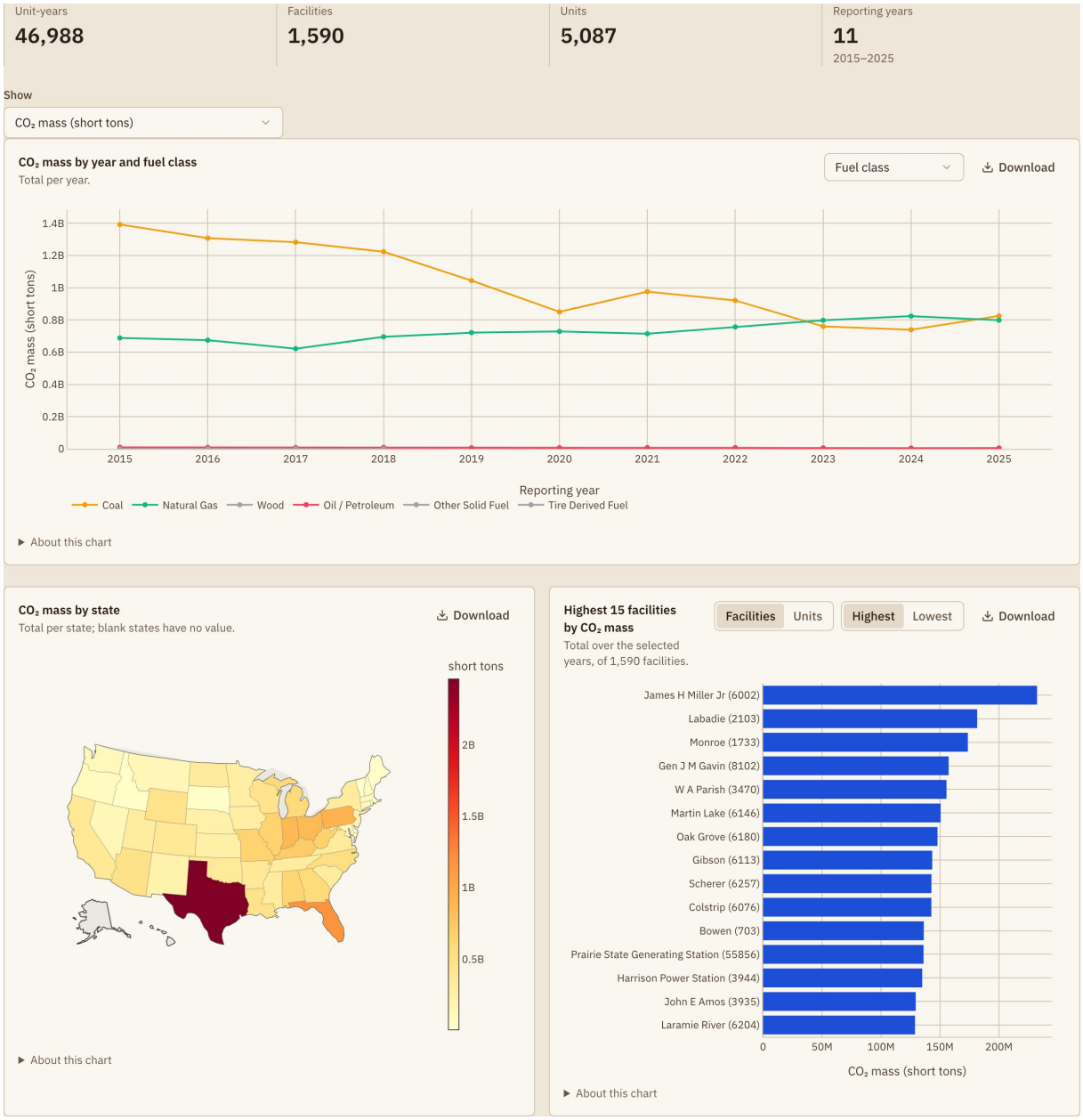


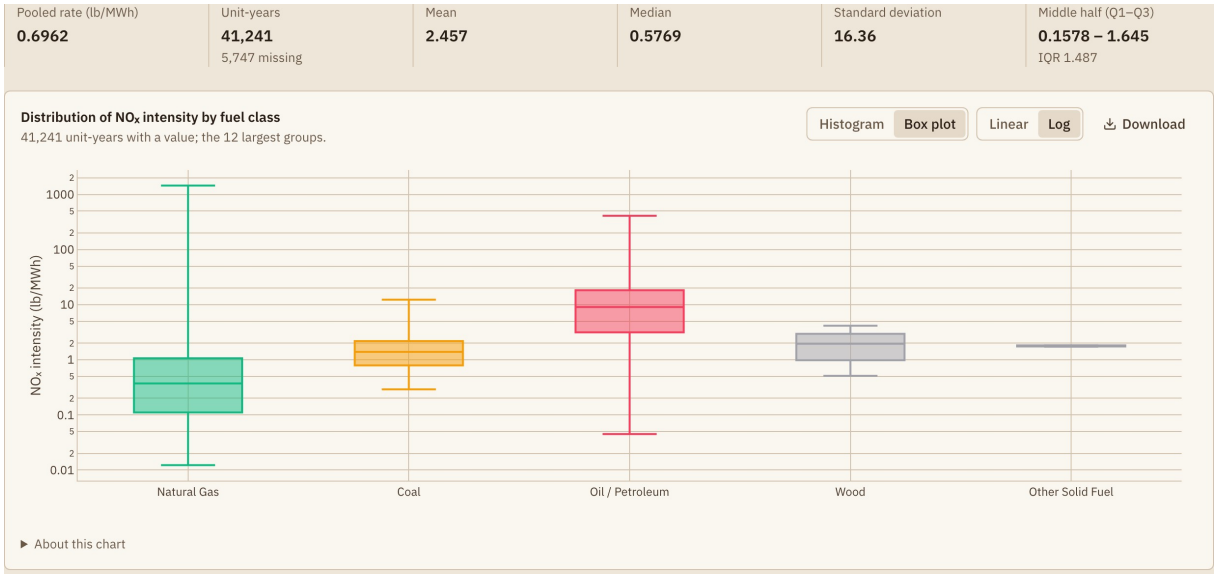
Figure 1: Overview tab for 2015–2025 (46,988 unit-years): counts, the CO_2 trend by fuel class, CO_2 by state, and the fifteen largest facilities.

2.2 Derived indicators

Twenty-nine variables are available: seven measured values, six derived rates, eight year-over-year changes, and eight TRACI indicators (Table 5). Derived and TRACI values are computed from the stored records each time an analysis runs, so they always match the current data. Section 5 gives the definitions and the missing-value rules.

2.3 Descriptive statistics

The Distribution tab (Figure 2) describes one variable over the selection: count, missing count, minimum, maximum, mean, median, standard deviation, first and third quartiles, interquartile range (IQR), and the 5th, 10th, 90th, and 95th percentiles, plus the variable's total, mean, or pooled rate over the whole selection. Six headline values come first, then a histogram or box plot, then one sortable table with a row for the whole selection and, with a grouping field, a row per group. It also lists *unusual observations*: unit-years beyond Tukey's far-out fences, $Q_1 - 3\,\mathrm{IQR}$ and $Q_3 + 3\,\mathrm{IQR}$, with the count on each side and the ten most extreme. The four example questions of the specification (average CO_2 by state, median NO_x intensity by fuel, total SO_2 by year, and average heat rate by unit type) are one-click examples.



Statistics by fuel class

NO_x mass × 2,000 ÷ gross load. Missing when either is missing or gross load is 0. 6 of 6 groups, largest pooled rate first. Missing values are left out, never counted as 0. Click a column to sort.

Download

Fuel class	Pooled rate	Count	Missing	Mean	Median	SD	Min	Q1	Q3	Max	IQR	P5	P10	P90	P95
All selected	0.6962	41,241	5,747	2.457	0.5769	16.36	0	0.1578	1.645	2,087	1.487	0.05016	0.06618	5.305	10.76
Other Solid Fuel	1.806	5	3	1.804	1.805	0.09801	1.705	1.725	1.833	1.951	0.1081	1.709	1.713	1.904	1.927
Oil / Petroleum	1.549	3,245	980	13.15	9.078	16.7	0.04029	3.145	18.33	411.3	15.19	0.339	1.138	26.77	35.73
Wood	1.472	120	235	1.916	1.162	1.104	0.5115	0.9583	2.865	4.156	1.907	0.5737	0.8108	3.447	3.756
Coal	1.358	5,143	1,228	1.666	1.35	1.232	0	0.7881	2.152	18.38	1.364	0.4965	0.5853	3.195	3.872
Natural Gas	0.2627	32,728	3,291	1.523	0.3653	17.23	0.01037	0.1104	1.062	2,087	0.9513	0.04719	0.05879	3.018	5.854
Tire Derived Fuel	–	0	10	–	–	–	–	–	–	–	–	–	–	–	–

Figure 2: Distribution tab: NO_x intensity over 2015–2025, overall and by fuel class (box plot on a log scale).

2.4 Interactive visualization

Seven chart types are drawn with Plotly.js [8] over the same selection, each on the tab whose question it answers: on Overview, a trend of any variable per year with one line per group, a state choropleth, and the top or bottom fifteen facilities or units; on Distribution, histograms and box plots, optionally on a log scale and split by group; on Relationships, scatter plots of any two variables, colored by group, and a Pearson or Spearman correlation heat map of eleven variables; and on Compare groups, bars for every group. The Phase 1 Map page still shows facility locations. Every chart has a descriptive title, axis titles with units, a legend, tooltips with values and counts, zoom, PNG export, and a *Download* button for its data, image, and metadata. A chart whose query fails says so and offers to try again. Fuel classes keep the colors they have on the Phase 1 map, and the models have colors of their own. For large selections, distributions use an even sample of at most 20,000 values and scatter plots at most 4,000 points, and the chart says so; statistics and correlations always use every value. Figure 3 shows the two chart types not shown elsewhere in this report, a histogram and a scatter plot; the trend, state map, and ranking are in Figure 1, the box plot in Figure 6, group bars in Figure 7, and the correlation heat map in Figure 8.

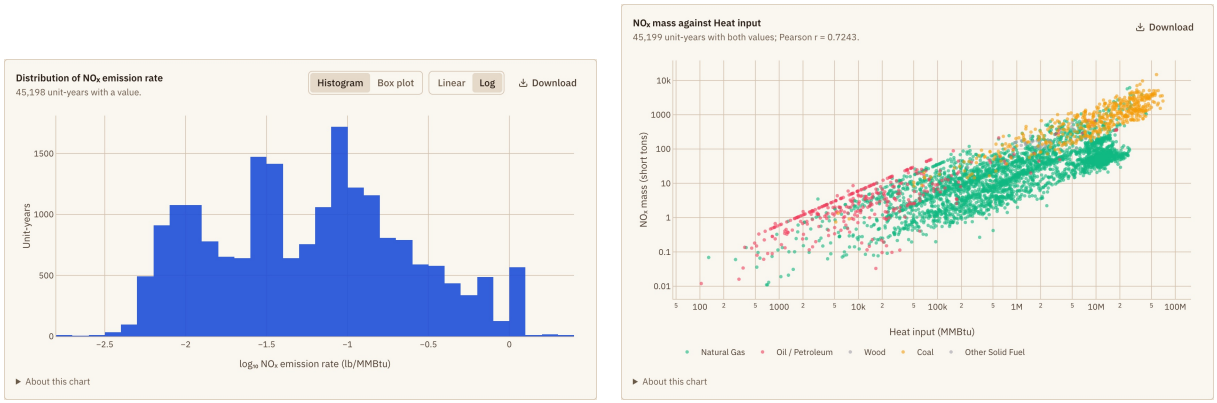


Figure 3: Left: histogram of the NO_x emission rate, 2015–2025, on a log scale (Distribution tab); the rate spans more than three orders of magnitude, with several peaks. Right: NO_x mass against heat input on log scales, colored by fuel class (Relationships tab). Since mass is rate times heat input, each fuel forms a band of slope about one, set higher or lower by its emission rate.

2.5 Comparative analysis

The Compare groups tab first draws the variable for every group of the chosen field, then compares two to eight of them: facilities or units (found with the search box), states, fuels, unit types, NO_x or SO_2 control classes (for example units with and without SCR), or reporting years (for example 2015 against 2024). For each group it shows the pooled value of the variable, its rank among the compared groups, its absolute and percentage difference from the first group, the count, the mean with a 95 % confidence interval, the median, and the standard deviation. It runs Welch's t-test and the Mann–Whitney U test for two groups, and one-way ANOVA and the Kruskal–Wallis test for any number, and states the result in plain language with its caveats (Figure 7). It opens on units with SCR against units with no NO_x control. The Phase 1 comparison of two to four individual facilities or units is still available from the Explore page, and its *Analyze these* button opens the same selection here, with every year and the tests. Likewise a facility's details link to Analyze and a unit's details to Predict.

2.6 Environmental and health impact indicators

The TRACI impacts tab converts the selected emissions into four TRACI 2.2 impact categories (global warming, acidification, particulate-matter human health, and smog formation), grouped by any field. Each category is shown in its own equivalent unit, as a total or per MWh of gross load, and categories are never added together. Each category's chart shows its own fifteen largest groups. The full results table and the factors used are in collapsed sections; the factors are listed with TRACI version, substance, CAS number, and unit, and the formula of each category. It also states that the results are potential impacts for comparison, not measured harm (Figure 9).

2.7 Prediction

The Models tab opens on the test error of the three registered models (baseline, Ridge regression, and decision tree) with 95 % bootstrap intervals, then compares them on the validation and test years; plots predicted against actual NO_x for the test years; and shows the tree's feature importances or the Ridge coefficients (Figure 10). A collapsed section gives the leakage rules and each model's record: inputs, settings, the years it was fit and tuned on, saved file, software versions, and training date.

2.8 Sensitivity and scenario analysis

The What-if tab starts from a baseline unit-year, chosen with the search box, a year, and a unit, and shows its inputs, its actual NO_x, and each model's prediction. The model control selects Ridge, the tree, or both, which are then drawn side by side. Below the baseline, the sensitivity analysis varies one input from a minimum to a maximum with a chosen step, given in the input's own unit or as a percentage of its baseline value, and holds all other inputs fixed. It draws the prediction as a curve with the baseline marked and the region outside the training data shaded (Figure 11), and a collapsed table gives each input value with its prediction and the absolute and percentage change. The scenario analysis on the same page compares the baseline with five named changes: gross load +10 % and heat input −5 % (both percentages editable), adding or removing SCR, switching the primary fuel, and a combined high-efficiency case (Figure 12).

2.9 Saving and exporting results

Every chart and the statistics, impact, and model-comparison tables have a *Download* button. Each downloads a ZIP file with the data as CSV, the chart as PNG when there is one, and a `metadata.json` file. The comparison, sensitivity, scenario, and predicted-against-actual results are exported through their charts. The metadata records the export time and page URL; the software version and Git commit; a snapshot of the database (unit-years, datasets, and time of the last import), which serves as the dataset identifier; the filters and tab choices; the definition, unit, and aggregation of each variable shown; the TRACI factors for impact results; and, on the Predict page, the model's inputs, settings, fit and tuning years, metrics, saved file, software versions, and the baseline unit-year. The full analytical dataset is exported by the server (`/api/export?type=analysis`) in the same format, with the filters in place of the URL. Section 7.4 shows an example.

2.10 Requirement traceability

Table 1 maps each requirement of the Phase 2 specification to the function that meets it.

Table 1: Phase 2 requirements and where they are met.

Specification	epaData	Status
§1 Trends, relationships, unusual observations	Trend and correlation charts; Tukey far-out fences on the Distribution tab	Met
§3 CAMPD multi-year data	2015–2025 API years (and older uploads) in `annual_records`	Met
§3 TRACI version, pollutant, category, factor, unit stored	`traci_factors` table	Met
§4.1 Filters: years, state, facility, unit, fuel, unit type, controls, program, minimum operating time or load	Explore filter bar on Analyze	Met
§4.1 Record, facility, unit, year counts; missing values; variables	Overview tab	Met
§4.1 Reproducible through URL or saved configuration	URL parameters; filters in exported metadata	Met (URL)
§4.2 Count, missing, min, max, mean, median, SD, quartiles, IQR, percentiles; grouped	Distribution tab	Met
§4.3 CO_2, SO_2, NO_x intensity; heat rate; year-over-year change; utilization	Derived variables; capacity factor	Met
§4.3 Division by zero and missing values stay missing	Variable rules; unit tests	Met
§4.4 Line, bar, histogram, box, scatter, correlation heat map, rankings	Overview, Distribution, Relationships, and Compare groups tabs	Met
§4.4 State or facility-location maps	State choropleth; Phase 1 Map page	Met
§4.4 Titles, units, legends, tooltips, filters, image and data download	Shared `Chart` component	Met
§4.5 Facilities, units, states, fuels, unit types, with and without control, periods	Compare groups tab; Phase 1 compare dialog	Met
§4.5 Statistics, differences, rankings, charts, normalized indicators, tests, plain language	Compare groups tab	Met
§4.6 Short tons to kg; exact pollutant match; sum within category; per MWh; factors recorded	TRACI impacts tab; `impactVariables`	Met
§5.1 One prediction target	Annual NO_x mass per unit-year	Met
§5.2 Candidate features; no target leakage	Section 5.5; ablation	Met
§5.3 Baseline and at least two models	Median baseline, Ridge, decision tree; persistence as a reference (Q4)	Met
§5.4 Chronological split; MAE, RMSE, R^2, MAPE	2016–21 / 2022–23 / 2024–25; 95 % bootstrap intervals on test metrics	Met
§6.2 Model, baseline unit, input, minimum, maximum, step; value, prediction, changes, curve	What-if tab	Met
§6.3 Named scenarios with range warnings	What-if tab	Met

Specification	epaData	Status
§6.4 Curve, baseline marks, scenario bars, labels, out-of-range shading	What-if tab	Met
§6.4 Same units; image or data export	Shared `Chart` component	Met
§6.4 Response of impact indicators	Not built; predictions are NO_x mass only	Not met (optional)
§7 Export filtered dataset	`/api/export?type=analysis`	Met
§7 Export statistics and comparison results	Statistics table; comparison chart	Met
§7 Export model evaluation and predictions	Model-comparison table; predicted-against-actual chart	Met
§7 Export sensitivity and scenario results	Sensitivity and scenario charts	Met
§7 Export chart images or data	PNG and CSV in every chart's ZIP	Met
§7 Metadata for reproducibility	`metadata.json` in every ZIP; a unit test checks each of the six items	Met
§8 Steps 5 and 10: pandas analysis; Flask pages or JSON	Statistics in TypeScript, checked against NumPy and SciPy; tRPC procedures (Section 3)	Changed
§8.1 Flask, Jinja2, SQLAlchemy, pandas, scikit-learn, joblib, pytest	Next.js, React, Drizzle; pandas, scikit-learn, joblib, and pytest offline	Changed
§12.3 Instructions, migrations, environment template, tests, fixtures, no keys	README, migration 0009, `.env.example`, 81 + 10 tests, 100 KB fixture	Met

2.11 Planned functions not completed or changed

Table 2 compares the Phase 2 plan with the delivered system.

Table 2: Plan versus delivered system.

Planned	Delivered	Reason
Store derived rates as columns	Computed when an analysis runs	Always consistent with the records; no backfill after an import.
Eutrophication from NO_x	Not provided	The TRACI 2.2 workbook has no eutrophication column.
Partial impact when a pollutant is missing	Category missing for that unit-year	A partial sum would understate the impact without saying so.
State and capacity factor as model inputs	Left out	State is sparse within many fuel and unit-type combinations; capacity factor repeats gross load and nameplate capacity.
Saved analysis configurations	URL parameters only	The URL already reproduces a view, so a saved configuration would be a bookmark.
Fully custom scenarios	Five named scenarios; two percentages editable	Time. The sensitivity analysis covers any single input.
Impacts of the predicted NO_x	Not built	Optional in §6.4; the scenarios report NO_x mass.
Residual plot	Predicted against actual only	The predicted-against-actual chart shows the same errors.
Gradient-boosting model	Not built	Time. The reformulations in Q4 suggest the inputs, more than the model class, limit accuracy.
O9 multi-indicator scoring	Not built	Not part of the Phase 2 specification.

3 System design

3.1 Architecture

Phase 2 keeps the Phase 1 stack: Next.js 16 with React 19, tRPC procedures for typed calls from the browser, Drizzle ORM, and SQLite. The specification recommends Flask, with pandas and scikit-learn for the analysis; the Phase 1 report explained how Flask, Jinja, and SQLAlchemy map onto the stack used. We extended the Phase 1 application instead of porting it to Flask: a port would have spent the phase re-implementing retrieval, upload, validation, and search rather than building the analysis, and the specification allows React for dashboards with extensive client-side state, which the Analyze and What-if pages are. Python still does what the specification assigns to it, training and evaluating the models with pandas and scikit-learn. For the analysis we chose a hybrid (Figure 4). Statistics, indicators, and TRACI impacts are computed in TypeScript on the server, next to the database. Models are trained offline in Python with pandas and scikit-learn [7], and each trained model is stored in the database with its preprocessing and parameters as JSON, which a small TypeScript module replays exactly. Predictions, sensitivity curves, and scenarios therefore need no Python process at request time and also run on the hosted build, where Python is not available.

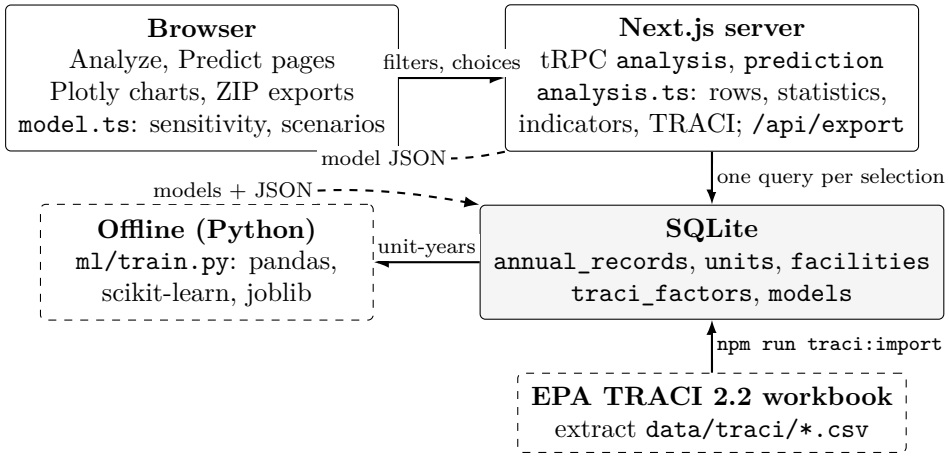


Figure 4: Phase 2 architecture. Solid arrows run on every request or import; dashed arrows carry the trained models from the offline pipeline to the browser.

3.2 Life cycle of an analysis

When a tab needs a result, the browser calls a tRPC procedure with the filters and the tab's choices. The server turns the filters into the same SQL conditions the Explore page uses (one shared function, `unitYearConditions`) and selects the matching unit-years, joined with their unit, their facility, and the same unit's previous year, which the year-over-year variables need. The rows are cached for five minutes, keyed by the filters and the time of the latest import, because one page usually asks for several views of the same selection. The variables are evaluated in memory and only the result (a table, chart points, or a matrix) is returned. Section 8.4 gives the measured times.

3.3 Database additions

Migration 0009 adds two tables. `traci_factors` holds one row per TRACI version, pollutant, medium, and impact category, with the TRACI substance name, CAS number, factor, unit,

and the workbook column it was read from; a unique index prevents duplicates. It is loaded from a six-row extract of the EPA workbook (`data/traci/traci_2_2_air.csv`), whose source, download date, and checksum are recorded next to it. `models` holds one row per trained model: name and kind, target, inputs, tuned settings, training, validation, and test years, metrics, the JSON artifact used for prediction, the path of the saved scikit-learn object, and the software versions and Git commit that produced it. The committed database contains six factors and three models. Column-level documentation is in `DATABASE_BREAKDOWN.md`.

3.4 Data flow and reproducibility

A Phase 2 result can be traced back through five steps. CAMPD records enter the database through retrieval or upload, as in Phase 1, and each record keeps the dataset that last wrote it. `npm run traci:import` loads the factor extract, whose checksum ties it to the EPA workbook. `npm run ml:train` reads the unit-years, trains and evaluates the models, and writes them with their metrics and software versions to the `models` table, together with the fixture used by the parity test. An analysis evaluates the variables for the selected unit-years. An export writes the result with the filters, variable definitions, factors or model record, the Git commit, and the database snapshot. Opening the recorded URL on a database with the same snapshot reproduces the same numbers.

4 Technical details

4.1 Languages

TypeScript implements the interface, the server, the statistics, the TRACI calculation, and model inference. Python implements the offline training pipeline (`ml/train.py`) and, as in Phase 1, the upload parser. SQL is mostly generated by Drizzle; the selection of unit-years with their previous year is a self-join written as a Drizzle query.

4.2 Frameworks and libraries

Table 3 lists what Phase 2 adds to the Phase 1 stack. The committed models were trained with Python 3.15 and the versions in the table. The minimum versions in `requirements.txt` allow the Python 3.9 documented in the README. On Python 3.9.25 with the newest versions it installs (pandas 2.3, NumPy 2.0, scikit-learn 1.6), the pipeline tests pass and retraining gives the same Ridge and baseline models; the tree's test MAE differs by 0.0001 t, and no reported value changes at the precision shown. NumPy 2.0 on macOS prints spurious overflow warnings from the bootstrap's matrix products; on the same inputs the bootstrap gives identical intervals on both versions.

Table 3: Libraries added in Phase 2.

Component	Role in epaData
Plotly.js 3 (`plotly.js-dist-min`)	Every chart: zoom, tooltips, box plots, heat maps, choropleth, PNG export. Loaded only when a chart is first shown.
fflate	Builds the ZIP exports in the browser and on the server.
pandas 3.1, NumPy 2.5	Feature extraction and preprocessing in `ml/train.py`.
scikit-learn 1.9.1	Ridge regression, decision tree, metrics.
joblib	Saves the fitted scikit-learn objects (`ml/models/*.joblib`).
SciPy 1.18	Reference values for the statistics tests.
pytest	Tests of the training pipeline (`ml/test_pipeline.py`).

4.3 External services

Vercel hosts the application at `https://cs396-project.vercel.app`. The hosted build is read-only (`READ_ONLY=true`): Upload and Retrieve show a notice to run the application locally, while Analyze and Predict work fully because they only read the committed database. The state choropleth loads the U.S. state outlines from Plotly's content network (`cdn.plot.ly`) at run time. Phase 2 adds no API keys; `.env.example` is unchanged.

4.4 Scripts and configuration

Table 4 lists the scripts added in Phase 2. The Phase 1 scripts are unchanged.

Table 4: Scripts added in Phase 2.

Command	Purpose
`npm run traci:import`	Loads `data/traci/traci_2_2_air.csv` into `traci_factors`.
`npm run ml:train`	Trains, tunes, and evaluates the models; writes `ml/models/`, the `models` rows, and the parity fixture.
`npm run test:py`	Runs the pytest tests of the training pipeline.
`npm test`	Runs all 61 TypeScript tests, including the Phase 2 ones.
`npm run report`	Builds this report with Tectonic.

5 Methods

5.1 Variables and missing values

Table 5 defines the derived variables. A value is *missing* when CAMPD did not report it, or when an input of a derived value is missing or its denominator is zero or negative. Missing values are left out of statistics and counted separately. They are never treated as zero, which would pull every mean and intensity down.

Table 5: Derived variables (computed per unit-year).

Variable	Unit	Definition
CO_2, SO_2, NO_x intensity	lb/MWh	mass (short tons) × 2,000 ÷ gross load
NO_x emission rate	lb/MMBtu	NO_x mass × 2,000 ÷ heat input
Heat rate	MMBtu/MWh	heat input ÷ gross load
Capacity factor	%	gross load ÷ (nameplate MW × hours in the year) × 100; 8,784 hours in leap years; values above 100 % are kept
Change (load, CO_2, SO_2, NO_x)	MWh or tons	this year minus the same unit's previous year
Change (%)	%	(this year − previous year) ÷ previous year × 100
TRACI impact, category c	kg eq	$\sum_i \text{mass}_{i,\text{kg}} \times \text{CF}_{i,c}$ (Section 5.4)
TRACI impact per MWh	kg eq/MWh	impact ÷ gross load

Combining groups. Each variable has one rule for combining unit-years into a group value. Masses, energy, changes in tons, and impacts are summed. Operating time and nameplate

capacity are averaged per unit-year. Rates are *pooled*: the sum of the numerators divided by the sum of the denominators, over the unit-years where the rate is defined. A state's CO_2 intensity is therefore its total CO_2 divided by its total gross load, not the mean of its units' intensities. The mean would give a peaking turbine that ran ten hours the same weight as a baseload coal unit.

5.2 Descriptive statistics and correlation

Quantiles use linear interpolation between order statistics, $Q(p) = x_{(\lfloor h \rfloor)} + (h - \lfloor h \rfloor)(x_{(\lfloor h \rfloor + 1)} - x_{(\lfloor h \rfloor)})$ with $h = (n-1)p$, which is Hyndman and Fan's type 7 [4] and NumPy's default. The standard deviation is the sample standard deviation $(n-1)$. Pearson's r and Spearman's ρ (Pearson's r of average ranks) use the unit-years where both variables are reported.

5.3 Comparisons and tests

The 95 % confidence interval of a group mean is $\bar{x} \pm t_{0.975,\,n-1}\, s/\sqrt{n}$. Two groups are compared with Welch's t-test [11], which does not assume equal variances, and with the Mann–Whitney U test [6] (normal approximation with tie correction), which asks whether one group tends to have larger values and is robust to the heavy tails of emissions data. Three or more groups are compared with one-way ANOVA and the Kruskal–Wallis test [5]. The t, F, χ^2, and normal distribution functions are computed in TypeScript from the regularized incomplete beta and gamma functions and checked against SciPy [10].

Two assumptions limit these tests, and the page states both. First, the groups differ in more than the attribute compared (size, age, fuel, hours run), so a difference is an association, not an effect of that attribute. Second, the tests treat every unit-year as independent, but the same unit appears in several years, so the p-values are smaller than they should be.

5.4 TRACI impact assessment

CAMPD supplies the inventory, the mass of each pollutant a unit emitted in a year. TRACI supplies characterization factors [1, 9]: the potential contribution of one kilogram of a substance emitted to air to an impact category. The two are joined by a fixed mapping from CAMPD pollutant to TRACI substance, never by approximate name matching. CAMPD's CO_2 maps to *CARBON DIOXIDE* (CAS 124-38-9), SO_2 to *SULFUR DIOXIDE* (CAS 7446-09-5), and NO_x to *NITROGEN OXIDES*, which has no CAS number in TRACI. TRACI also lists *NITROGEN DIOXIDE*, with a smog factor of 16.85 kg O_3 eq/kg against 24.79 for nitrogen oxides. CAMPD reports NO_x, so the nitrogen-oxides row is the right one. Table 6 lists every non-empty air factor of the three substances in the TRACI 2.2 workbook.

Table 6: TRACI 2.2 factors used (per kg emitted to air).

Category	CO_2	SO_2	NO_x
Global warming (kg CO_2 eq)	1	—	—
Acidification (kg SO_2 eq)	—	1	0.7
Human health, particulate (kg $PM_{2.5}$ eq)	—	0.061111	0.0072222
Smog formation (kg O_3 eq)	—	—	24.793590

For each unit-year, CAMPD masses are converted from short tons to kilograms and multiplied

by their factors, summing only within a category:

$$\text{Mass}_{i,\text{kg}} = \text{Mass}_{i,\text{short ton}} \times 907.18474, \tag{1}$$

$$\text{Impact}_c = \sum_i \text{Mass}_{i,\text{kg}} \times \text{CF}_{i,c}, \tag{2}$$

$$\text{Impact intensity}_c = \text{Impact}_c \,/\, \text{Gross load}_{\text{MWh}}. \tag{3}$$

Categories keep their own units and are never added together. If a pollutant that contributes to a category was not reported, that category is missing for the unit-year, while categories that do not need the pollutant are still computed: a unit-year without SO_2 has no acidification result but does have a smog result. Group results are sums of unit-year results, and group intensities are pooled as above.

Worked example. In 2024 the four units of Ghent (facility 1356, Kentucky) reported 11,399,877.916 short tons of CO_2, 9,157.169 tons of SO_2, 6,987.671 tons of NO_x, and a gross load of 11,580,029.2 MWh. Acidification is $(9{,}157.169 \times 1 + 6{,}987.671 \times 0.7) \times 907.18474 = 14{,}048.539 \times 907.18474 = 12{,}744{,}620$ kg SO_2 eq, or 1.1006 kg SO_2 eq per MWh. Global warming is $11{,}399{,}877.916 \times 907.18474 = 1.0342 \times 10^{10}$ kg CO_2 eq; particulate human health is 553,447 kg $PM_{2.5}$ eq; smog formation is 1.5717×10^8 kg O_3 eq. The TRACI impacts tab, grouped by facility for 2024, shows the same values.

5.5 Prediction

Target. The models predict a unit's annual NO_x mass in short tons. We chose NO_x over CO_2 because CO_2 is close to a fixed multiple of heat input for a given fuel, so a CO_2 model would mostly learn that multiple. NO_x depends on combustion conditions and controls, so the controls carry information, and every scenario in the specification (changed load, changed heat input, changed control) affects the prediction.

Inputs. Heat input, gross load, and operating time of the same year; nameplate capacity; unit age (reporting year minus the year of commercial operation); the unit's NO_x rate (lb/MMBtu) and operating time in the previous year; primary fuel and unit type; and five NO_x control flags (SCR, SNCR, low-NO_x burners, water, steam, or ammonia injection, and overfire air), read from the controls reported that year.

Leakage. No input is the target or is computed from it: same-year NO_x mass, NO_x rate, and NO_x intensity are excluded. The previous year's NO_x rate is allowed because it is known before the year starts. Same-year heat input, gross load, and operating time are not known in advance, but they are what the scenarios change, so they are kept. To show how much accuracy they provide, every model is also fitted without them (the ablation column of Table 7).

Split. Unit-years from 2016 (the first year with a previous year) to 2025 with a reported NO_x mass are split by year: 25,116 training unit-years (2016–2021), 7,970 validation unit-years (2022–2023), and 7,755 test unit-years (2024–2025). Settings are chosen on the validation years; the chosen models are then refit on 2016–2023 and evaluated once on 2024–2025. A random split was not used because it would let a model learn a unit's 2024 emissions from its 2025 record.

Models. The baseline predicts the median NO_x of training unit-years with the same primary fuel and unit type, or the overall median for unseen combinations. Ridge regression [3] is fitted

to $\log(1 + \mathrm{NO}_x)$, because $\mathrm{NO_x}$ spans six orders of magnitude, with the penalty α chosen from $\{0.01, 0.1, 1, 10, 100\}$; its predictions are transformed back and floored at zero. The decision tree [2] is fitted to tons, with the maximum depth chosen from $\{4, 6, 8, 10, 12, 16, 20, 24\}$ or unlimited and the minimum leaf size from $\{5, 20, 50\}$. Validation selected $\alpha = 10$ and a maximum depth of 20 with at least 20 unit-years per leaf. An earlier grid stopped at depth 16 and chose its top value, so it was extended. The extension changes nothing: with 20 unit-years per leaf the tree stops growing at depth 18 by itself, so limits of 20, 24, and none give the same tree, and its validation MAE differs from the depth-16 tree's by 0.001 tons.

Persistence reference. A model can only be judged against a fair alternative. Besides the required median baseline, `train.py` computes a *persistence* reference: the unit's previous-year $\mathrm{NO_x}$ rate times this year's heat input, divided by 2,000, or the median baseline when either is missing (1.4 % of test unit-years). It uses only inputs the models also see. In the ablation, where same-year heat input is unknown, it predicts last year's tons. It is reported with the metrics (`ml/models/metrics.json`) but not registered as a model or shown in the application.

Metrics. Mean absolute error (MAE), root mean squared error (RMSE), coefficient of determination (R^2), and mean absolute percentage error (MAPE). MAPE is computed only over unit-years of at least one ton, because for peaking turbines that emit a fraction of a ton a percentage error has no useful meaning. Each test metric has a 95 % percentile interval from 1,000 bootstrap samples. Because the test years hold up to two unit-years of each unit, which are not independent, each sample draws units with replacement and keeps all of a drawn unit's years (a cluster bootstrap). The same samples give an interval for the difference between a model's MAE and the persistence reference's.

Sensitivity and scenarios. A sensitivity curve changes one input of the baseline unit-year at each step, holds every other input fixed, and records the prediction, its absolute change, and its percentage change, $(\hat{y}_{\text{scenario}} - \hat{y}_{\text{baseline}})/\hat{y}_{\text{baseline}} \times 100$. An input is *outside the training data* when it is below the minimum or above the maximum of that input in the training years, or when its fuel or unit type was not seen in training. The check is per input, so a combination of individually ordinary values can still be unlike any training unit-year.

6 Results

All results use the reporting years 2015–2025 of the committed database unless stated otherwise.

6.1 Q1: emissions trends and the shift from coal to gas

Total $\mathrm{CO_2}$ from the selected units fell from 2,098 million short tons in 2015 to 1,635 million in 2025 (22 %), and the pooled $\mathrm{CO_2}$ intensity fell from 1,557 to 1,279 lb/MWh (18 %). Figure 5 shows where the decline came from. Coal units emitted 1,393 million tons in 2015 and 739 million in 2024, while gas units rose from 688 to 824 million. The two lines crossed twice: in 2023 gas units emitted more $\mathrm{CO_2}$ than coal units for the first time in the data, and in 2025 coal was ahead again (825 against 799 million). The intensity of each fuel changed little: coal units averaged 2,041 lb/MWh in 2015 and 2,036 in 2025, gas units 1,051 and 926. The sector's falling intensity is therefore mainly a change in the generation mix, not cleaner coal plants.

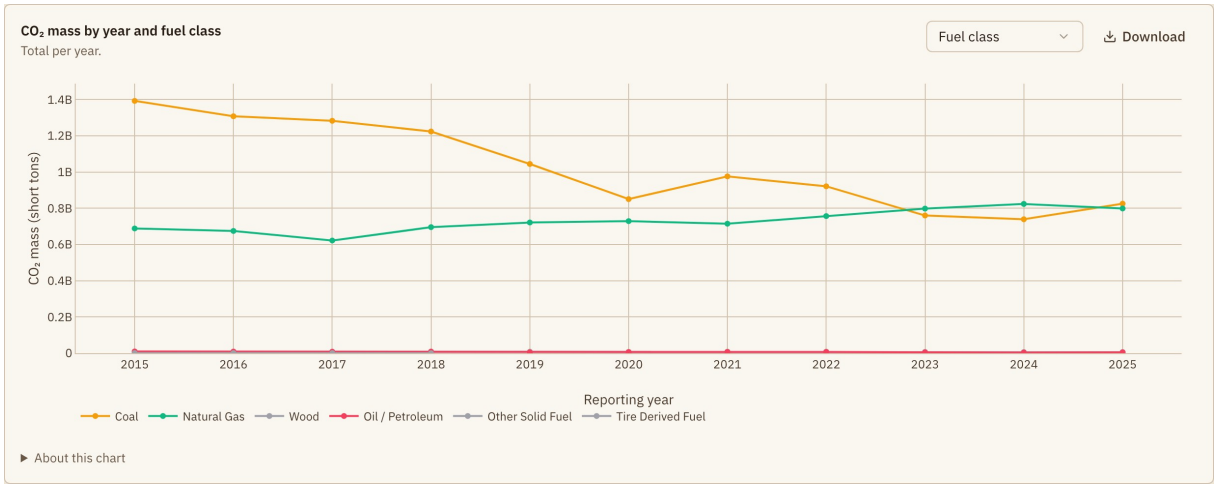


Figure 5: Trend chart: total CO_2 per year from coal and natural-gas units, 2015–2025.

6.2 Q2: NO_x rate and post-combustion control

Figure 6 shows the NO_x emission rate by fuel class on a log scale. Gas units have a median of 0.034 lb/MMBtu (IQR 0.013–0.096), coal units 0.136 (0.082–0.219), and oil units 0.574 (0.188–1.20), with wide overlap within each fuel.

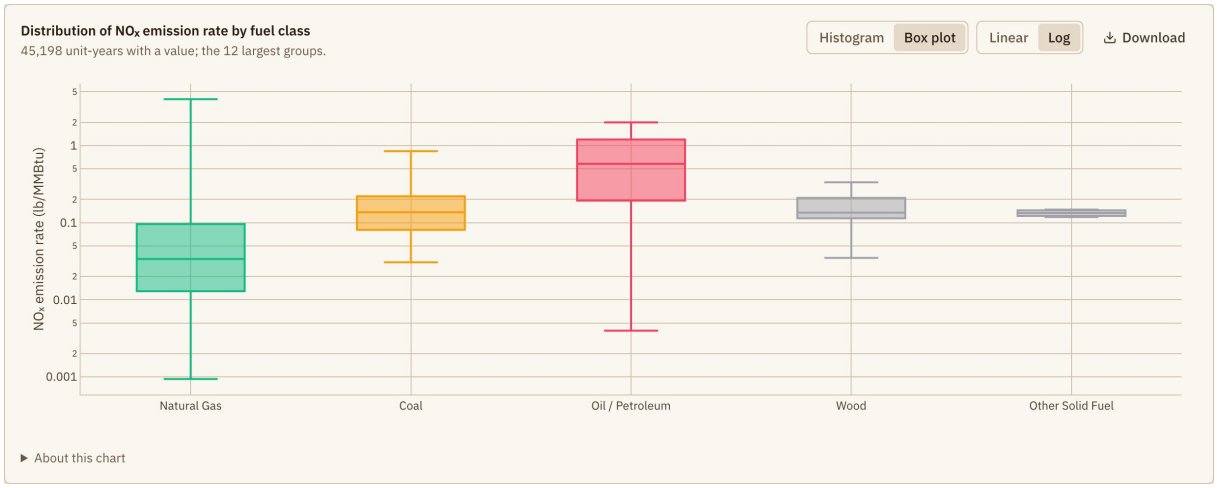


Figure 6: Box plot of the NO_x emission rate by fuel class (log scale).

In 2024, 1,564 unit-years list SCR among their NO_x controls and 571 list no NO_x control. The rate is defined (NO_x mass reported and heat input above zero) for 1,547 and 479 of them. Their pooled rates are 0.037 and 0.159 lb/MMBtu, so units without control emit at a 330 % higher rate; the medians are 0.012 and 0.305 (Figure 7). Welch's test ($t = -22.3$, $p < 0.001$) and the Mann–Whitney test ($z = -29.8$, $p < 0.001$) both reject equal groups. The difference is real, but not all of it is due to SCR, because the two groups are different fleets: 881 of the 1,564 SCR unit-years are gas combined-cycle units, while the 571 uncontrolled unit-years are mostly gas combustion turbines and oil units (326 gas and 212 oil). A fairer comparison holds fuel and unit type fixed, which the Compare groups tab supports by adding those filters.

Unusual observations. Over 2015–2025, 3,582 unit-years lie above the far-out fence for NO_x intensity (6.1 lb/MWh, against a median of 0.58). Of these, 3,491 are combustion turbines with a median operating time of 44 hours a year: peaking units whose start-ups emit NO_x

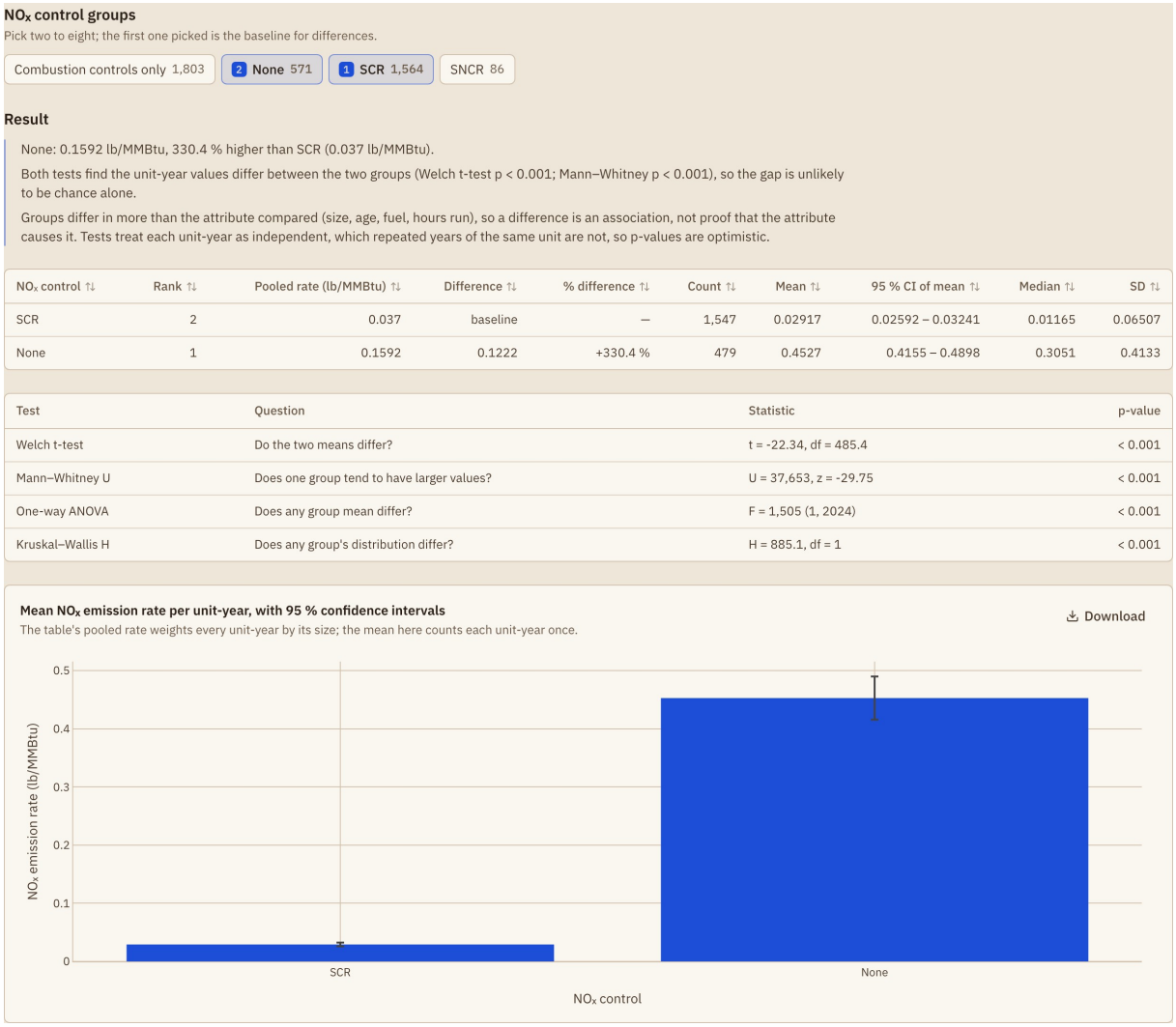


Figure 7: Compare groups tab: NO_x emission rate of units with SCR and with no NO_x control, 2024.

with almost no generation. They are real reports, not errors, and they are why the pooled rate (0.70 lb/MWh) is a better summary than the mean (2.46).

A misleading correlation. The Pearson correlation between CO_2 intensity and heat rate over the 37,718 unit-years reporting both is 0.9998, while Spearman's rank correlation is 0.765 (Figure 8). A few unit-years with tiny gross loads have intensities and heat rates thousands of times the norm. They dominate Pearson's r and barely affect the ranks, so Spearman's ρ is the better summary for these heavy-tailed variables. This is why the chart offers both.

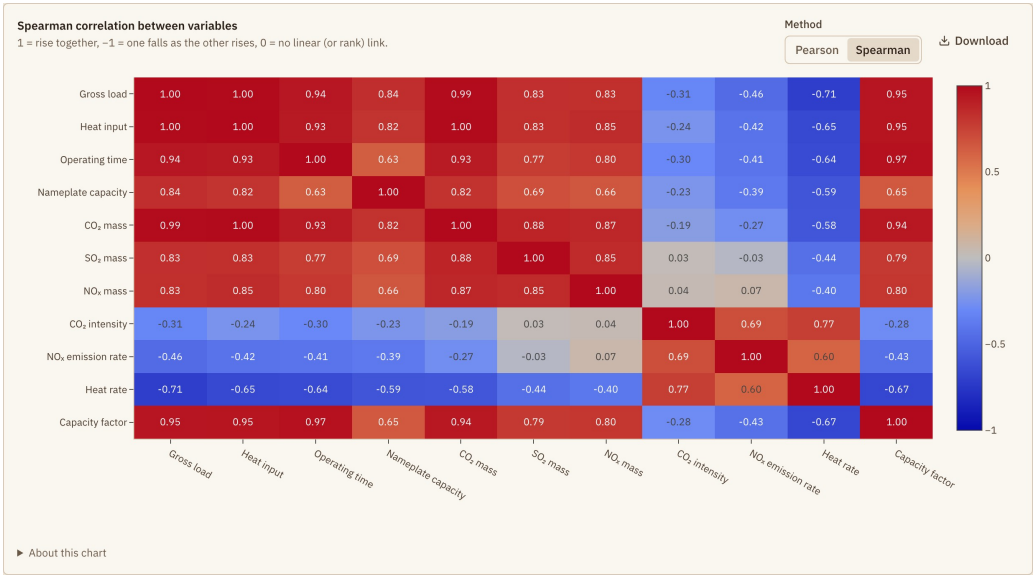


Figure 8: Spearman correlation heat map of eleven variables, 2015–2025.

6.3 Q3: impacts by state

In 2024 Texas contributed the most to every category: 1.91×10^{11} kg CO_2 eq of global warming and 1.34×10^{8} kg SO_2 eq of acidification, followed for acidification by Missouri (7.93×10^{7}) and Kentucky (5.65×10^{7}) (Figure 9). Per MWh the order changes: Nebraska (2.32 kg SO_2 eq/MWh), North Dakota (1.83), and Missouri (1.57) lead. Coal units produced 93 % of the reported gross load of Nebraska and North Dakota and 83 % of Missouri's, against 20 % in Texas, whose pooled SO_2 intensity was 0.51 lb/MWh against Nebraska's 3.95. Totals show where impacts occur; intensities show how clean a state's generation is.

6.4 Q4: prediction

Table 7 compares the models. Both trained models beat the median baseline by a wide margin on the test years: Ridge lowers the MAE by 47 % and the tree by 76 %, and the intervals do not overlap. The tree's training ($R^2 = 0.956$), validation, and test results are close, so it does not noticeably overfit despite its depth. Without same-year heat input, gross load, and operating time, validation R^2 falls from 0.853 to 0.704 for Ridge and from 0.967 to 0.837 for the tree. Most of the remaining accuracy comes from the previous year's NO_x rate, which reflects the unit's combustion technology and controls.

The persistence reference is more accurate than both models. Its test MAE is 18.5 tons (16.1–21.2), against 26.3 for the tree; the tree's MAE is 7.8 tons higher (6.3–9.4), and Ridge's 40.0 tons higher (35.8–44.8). Persistence also wins without same-year activity, where last year's tons reach



Figure 9: TRACI impacts tab: TRACI categories by state for 2024, each in its own unit (the 15 largest states per category).

a validation R^2 of 0.884 against the tree's 0.837. The reason is physical: NO_x mass is the emission rate times heat input, and a unit's rate changes little from one year to the next. A tree can approximate that product only in steps, and Ridge on the log scale gives the previous rate a linear rather than a logarithmic effect.

We tried three reformulations that build the product into the model, each tuned on the validation years and scored once on the test years (`ml/reformulations.py`): the persistence estimate as an extra input (test MAE: tree 21.3, Ridge 25.0 tons); predicting this year's rate and multiplying by heat input (tree 20.2, Ridge 44.8); and predicting a correction to the persistence estimate (tree 19.7, Ridge 22.4). None beats persistence: the inputs carry little information about *changes* in a unit's rate. One likely cause of such changes, a control installed or retired, is invisible to the models, which see this year's controls but not last year's, so they cannot tell a new control from one the previous rate already reflects.

The models are therefore not the best point forecast of next year's NO_x. What they add is a response to inputs that persistence ignores: persistence scales only with heat input, while the models also respond to load, controls, fuel, and unit type, which is what the What-if scenarios need. Those responses are associations learned across units, not measured effects, as the scenarios below show.

Table 7: NO_x models: validation (2022–2023) and test (2024–2025) results, with 95 % cluster-bootstrap intervals for test MAE and R^2. Ablation: validation R^2 without same-year heat input, gross load, and operating time. Persistence is a reference, not a registered model.

	Validation		**Test**				**Ablation**
Model	MAE (t)	R^2	MAE (t)	RMSE (t)	R^2	MAPE	R^2
Baseline (median)	114.2	0.474	109.4 99.3–121.0	366.9	0.489 0.443–0.538	143.6 %	0.474
Ridge ($\alpha = 10$)	57.9	0.853	58.5 52.7–64.9	213.5	0.827 0.784–0.870	49.3 %	0.704
Tree (depth ≤ 20, leaf 20)	26.5	0.967	26.3 23.4–29.7	113.9	0.951 0.937–0.963	20.9 %	0.837
Persistence (reference)	18.7	0.965	18.5 16.1–21.2	92.8	0.967 0.953–0.979	15.3 %	0.884

Sensitivity. Figure 11 varies the heat input of Ghent unit 2 in 2024, a 556 MW tangentially fired coal unit with low-NO_x burners and overfire air but no SCR. It emitted 4,146 tons of NO_x. Both models under-predict this unit-year: Ridge predicts 2,972 tons and the tree 2,738. The reason is in the inputs: the unit's NO_x rate rose from 0.227 lb/MMBtu in 2023 to 0.314 in 2024, and the models rely on the previous year's rate. Raising heat input by 50 % raises Ridge's prediction to 3,827 tons (29 %), an elasticity of about 0.6. The tree behaves differently: above about 33 million MMBtu its prediction stops rising (about 3,500 tons), well inside the training range. A tree's prediction is a step function: along this unit's path no split separates larger heat inputs, so more heat input changes nothing. Beyond 85.8 million MMBtu, the largest heat input in the training years, both curves enter the shaded region and their points fade.

Scenarios. Figure 12 applies the named scenarios to the same unit. The models move in the same direction when heat input falls by 5 % (Ridge −3.2 %, tree −9.1 %), and neither responds to a 10 % rise in gross load once heat input is held fixed. They disagree on controls and fuel. Ridge predicts that adding SCR would cut NO_x by 59 % and switching to pipeline gas by 69 %; the tree predicts no change for either. The tree splits mostly on heat input and the previous



Model	Settings	Val. MAE (t)	Val. R^2	Test MAE (t)	Test RMSE (t)	Test R^2	Test MAPE	Val. R^2, no same-year activity
Baseline: median by fuel and unit type	median of training years	114.2	0.4744	109.4 99.26–121	366.9 323.4–415.8	0.4887 0.4433–0.5385	143.6 %	0.4744
Ridge regression (log target)	$\alpha = 10$	57.88	0.8533	58.47 52.68–64.86	213.5 180.9–249	0.8269 0.7844–0.8696	49.35 %	0.7045
Decision tree	depth ≤ 20, leaves ≥ 20	**26.49**	**0.9671**	**26.31** 23.39–29.73	**113.9** 94.06–134.3	**0.9507** 0.9374–0.9632	**20.87 %**	**0.8373**

Figure 10: Models tab: test error with 95 % intervals and all metrics against the baseline, predicted against actual NO_x for the test years, and the Ridge coefficients.

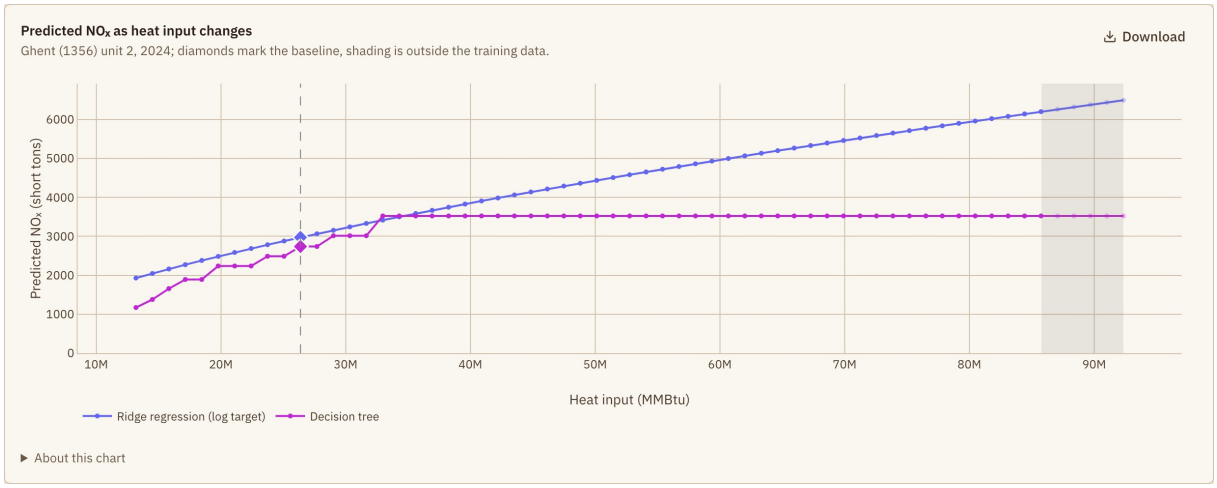


Figure 11: Sensitivity of the Ridge and tree NO_x predictions to heat input for Ghent unit 2, 2024, from $-50\,\%$ to $+250\,\%$. Shaded: outside the training data.

year's NO_x rate, which still describes the unit as it was, so a changed control flag rarely changes its path. Ridge applies one average SCR coefficient to every unit. Neither number is a causal estimate. When two reasonable models disagree this much, the data do not determine the effect, and the page says so.

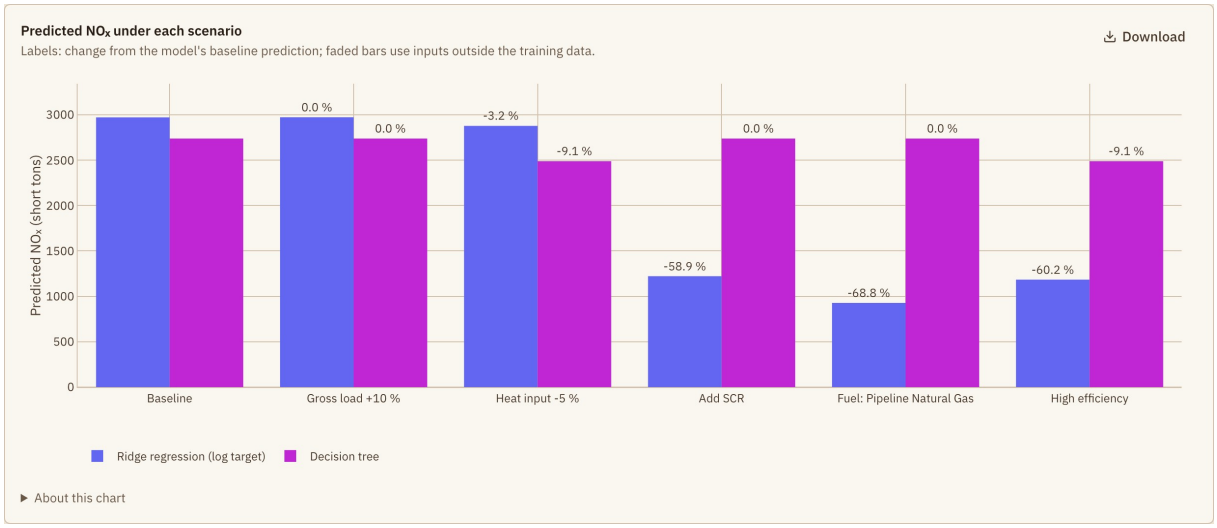


Figure 12: Scenarios for Ghent unit 2, 2024, under Ridge and the decision tree; labels give the change from each model's baseline prediction.

7 Highlighted features

This section describes four implementation choices in more detail, with the code behind them.

7.1 One variable catalogue for every analysis

Every measured value, derived rate, year-over-year change, and TRACI category is an object with a key, unit, definition, aggregation rule, and a function that computes its value for one unit-year (`buildVariables` in `src/lib/analysis.ts`). Statistics, charts, comparisons, impacts, exports, and the Overview tab's variable list all read this one list, so each is written once and

works for all 29 variables. Rates are built by one helper that also stores the numerator and denominator, which is what makes pooling possible (Listing 1).

```
/** A rate: null when either input is missing or the denominator is 0 (or negative). */
function ratio(base, num: Value, den: Value, scale = 1): Variable {
  return {
    ...base,
    aggregation: "ratio",
    ratio: { num, den, scale },
    value: (r) => {
      const n = num(r);
      const d = den(r);
      return n === null || d === null || d <= 0 ? null : (n / d) * scale;
    },
  };
}

/** A variable's value over a group, by its aggregation; null when no row defines it. */
export function aggregate(v: Variable, rows: readonly AnalysisRow[]) {
  let n = 0, sum = 0, num = 0, den = 0;
  for (const r of rows) {
    const x = v.value(r);
    if (x === null) continue;           // missing: skipped, never counted as 0
    n++;
    sum += x;
    if (v.ratio) {
      num += v.ratio.num(r)!;
      den += v.ratio.den(r)!;
    }
  }
  if (!n) return null;
  if (v.ratio) return den > 0 ? (num / den) * v.ratio.scale : null;
  return v.aggregation === "mean" ? sum / n : sum;
}
```

Listing 1: Rates and pooling (`src/lib/analysis.ts`, abridged).

7.2 TRACI impacts as ordinary variables

Each TRACI category becomes two variables built from the rows of `traci_factors`: the total and the per-MWh rate. Because the factors come from the database, the formula shown in the interface and written to export metadata is generated from the same rows used in the calculation (Listing 2).

```
const terms = factors.filter((f) => f.impactCategory === category);
const impact: Value = (r) => {
  let total = 0;
  for (const { p, factor } of parts) {   // one term per matched pollutant
    const tons = r[p.mass];
    if (tons === null) return null;     // a missing pollutant makes the category missing
    total += tons * SHORT_TON_KG * factor;   // SHORT_TON_KG = 907.18474
  }
  return total;
};
```

Listing 2: Impact of one category for one unit-year (`impactVariables`, abridged).

7.3 Prediction without Python on the server

`ml/train.py` implements its own preprocessing instead of scikit-learn's pipeline objects, so that every step can be written to JSON. Missing numeric inputs are replaced by the training median, three skewed inputs are log-transformed ($\log(1+x)$), numeric inputs are standardized with the training mean and standard deviation, fuel and unit type are one-hot encoded over categories seen at least 30 times, and the control flags are passed through. Ridge stores its coefficients and intercept; the tree stores its node arrays (children, split feature, threshold, and leaf value). `src/lib/model.ts` repeats the same steps.

One detail mattered. scikit-learn compares tree features as 32-bit floats, so the TypeScript version rounds each feature to single precision before comparing it with a threshold (Listing 3). Without this, a value that falls between a threshold's single- and double-precision forms takes the wrong branch. A parity test replays 200 test-year unit-years exported by the training script and requires each model's TypeScript prediction to equal scikit-learn's within 10^{-6}.

```ts
// scikit-learn trees compare float32 features against their thresholds.
const t = model.tree!;
let node = 0;
while (t.left[node] !== -1) {
  node =
    Math.fround(x[t.feature[node]!]!) <= t.threshold[node]!
      ? t.left[node]!
      : t.right[node]!;
}
return t.value[node]!;
```

Listing 3: Tree inference (`predict` in `src/lib/model.ts`).

7.4 Exports that can be reproduced

Exporting Kentucky, 2024 with Analyze's *Download all* gives a ZIP with a CSV of 90 unit-years (one column per variable) and the `metadata.json` shown, shortened, in Listing 4. The database snapshot and commit identify the data and code; the filters and variable definitions say what was selected and how each column was computed; the factor list makes every impact value checkable by hand.

```
{
  "title": "Analytical dataset",
  "exportedAt": "2026-10-31T18:00:52.610Z",
  "software": { "name": "epaData", "version": "0.1.0", "commit": "224f6f1" },
  "database": { "unitYears": 57414, "datasets": 153,
                "lastImport": "2026-10-11T04:38:52.000Z" },
  "filters": { "stateCode": "KY", "yearMin": "2024", "yearMax": "2024" },
  "unitYears": 90,
  "variables": [ { "key": "co2Intensity", "unit": "lb/MWh",
      "definition": "CO2 mass x 2,000 / gross load. Missing when ...",
      "aggregation": "ratio" }, ... ],
  "traciFactors": [ { "traciVersion": "2.2", "pollutant": "nox",
      "substance": "NITROGEN OXIDES", "impactCategory": "Smog formation",
      "factor": 24.793589743589738, "factorUnit": "kg O3 eq" }, ... ],
  "missingValues": "Blank cells are missing: not reported, or undefined ..."
}
```

Listing 4: `metadata.json` of an analytical-dataset export (shortened).

8 Testing and results

8.1 Approach

Each calculation was tested against an independent reference: statistics against NumPy and SciPy, TRACI results against hand calculation, TypeScript predictions against scikit-learn, and counts against hand-written SQL. Every page and chart was then used in a headless browser (Playwright with Chromium) on the development and production builds, at desktop and phone widths, checking for console errors and reading each screenshot. Downloaded ZIP files were unpacked and their CSV and metadata checked. On the hosted build we checked that Analyze and Predict load with the same numbers and that Upload refuses to write. `npm run validate` (formatting, linting, type checking, unit tests, and a production build) passes.

8.2 Unit tests

`npm test` runs 81 TypeScript tests in nine files: 80 pass and one is skipped because it needs a folder of raw CAMPD files that is not committed. Thirty-seven tests in four new files cover Phase 2:

- `stats.test.ts` (4 tests): quantiles and standard deviation against NumPy; Pearson and Spearman correlation, Welch's t, Mann–Whitney U, the confidence interval, ANOVA, Kruskal–Wallis, and the χ^2 tail against SciPy, to 10^{-9} for statistics and 10^{-6} for p-values; pairwise-complete correlation; constant inputs.
- `analysis.test.ts` (13 tests): each derived rate, and its missing value when an input is missing or a denominator is zero; capacity factor in leap and ordinary years; year-over-year changes with a missing or zero previous year; pooled rates as a ratio of sums; TRACI impacts against hand calculation, with categories kept apart and a missing pollutant affecting only its own categories; the factor extract (six factors, nitrogen oxides rather than nitrogen dioxide); the control classes; percentage formatting; the URL state (default year range, old tab links, unknown fields); and `findVariable`, which rejects an unknown variable key so that the server answers with an error (HTTP 400) instead of an empty result.
- `model.test.ts` (14 tests): the parity test (all three models on 200 test unit-years, within 10^{-6}); input derivation (unit age, control flags, and a previous rate that is missing when the previous heat input is zero); out-of-range detection; the direction of the response to heat input; readable feature names; and the What-if calculations: percentage change (undefined from zero), the sensitivity grid (baseline included once, a reversed range, a zero step, the point limit), the curve (only the varied input changes; absolute and percentage changes from the baseline prediction; values past the training maximum flagged), and the scenarios (each changes only what it names; coal and gas units get the opposite fuel and control changes; an unseen fuel is flagged).
- `export-bundle.test.ts` (6 tests): a ZIP reads back as the CSV, `metadata.json`, and PNG that went in; and the metadata of every kind of export (Analyze charts, the analytical dataset, and Predict) contains each of the six reproducibility items of §7, while metadata missing any one of them is reported with that item's name.

`npm run test:py` runs ten pytest tests of the training pipeline: the three periods do not overlap; no input is the target or derived from it; the input rules; the preprocessing specification gives the same matrix after a round trip through JSON; the baseline is the group median; MAPE ignores targets below one ton; persistence is the previous rate times heat input, with the baseline as fallback and last year's tons in the ablation; the bootstrap intervals contain the point estimates and detect a clearly better model; and the tree grid extends past its earlier limit.

8.3 Representative test cases

Table 8 lists end-to-end cases. Each expected value comes from an independent source, usually a hand-written SQL query against the committed database.

Table 8: Representative test cases.

Input	Expected (source)	Actual	Pass
Analyze with default filters (2015–2025)	46,988 unit-years, 1,590 facilities, 5,087 units (SQL)	Same	✓
Analyze: KY, 2023–2024	177 unit-years (SQL)	177	✓
Program CSNOX, 2015–2025	24,676 whole-code matches; a substring match would give 25,882 (SQL)	24,676	✓
`/analyze?tab=charts` (an old link) with no filter	Default range 2015–2025 applied; opens Overview	Same	✓
CO_2 intensity with gross load 0	Missing, not 0 (1,506 unit-years with CO_2 but no positive load)	Missing	✓
Impacts, Ghent, 2024	12,744,620 kg SO_2 eq; 1.1006 per MWh (hand calculation)	Same	✓
Compare 2024 NO_x rate, SCR vs no control	Pooled 0.037 and 0.159 lb/MMBtu over 1,547 and 479 unit-years (SQL)	Same	✓
Models tab, test years	MAE 109.4, 58.5, 26.3 t; intervals as in Table 7 (`metrics.json`)	Same	✓
Persistence, 7,648 test unit-years with a previous rate	MAE 18.286 t (SQL)	18.286 t	✓
Analyze request with an unknown variable	HTTP 400, “Unknown variable” (tRPC error contract)	Same	✓
TypeScript vs scikit-learn, 200 unit-years	Equal within 10^{-6} (parity fixture)	Equal	✓
Ghent unit 2, 2024, heat input +250 %	Outside training range (max 85.8 million MMBtu)	Warned, shaded	✓
Scenarios, Ghent unit 2, add SCR	Ridge −58.9 %, tree 0.0 % (direct model call)	Same	✓
Export dataset, KY, 2024	90 rows; metadata with filters, 29 variables, 6 factors; no §7 item missing	Same	✓
Upload or retrieve on the hosted build	Refused with a “run locally” notice	Refused	✓

8.4 Performance

Table 9 gives server-side times on an Apple-silicon laptop (Node.js 26) over the 2015–2025 selection of 46,988 unit-years, as the median of three runs. They exclude the network and chart drawing.

Table 9: Measured computation times for the 2015–2025 selection.

Operation	Time
Load the selection (first request, SQL query and join)	0.5 s
Load the same selection again (cache)	< 1 ms
Overview summary (missing counts of 29 variables)	219 ms
Statistics of NO_x intensity by fuel class	62 ms
Trend of CO_2 by year and fuel class	23 ms
Comparison of two groups with all tests	11 ms
Correlation heat map, 11 variables, Pearson	182 ms
Correlation heat map, 11 variables, Spearman	1.9 s
Sensitivity curve, 61 tree predictions (browser code)	0.1 ms
Analytical-dataset export, 2015–2025 (ZIP)	0.7 s

Most views take well under a second. The Spearman heat map is the exception: it ranks the values again for each of the 55 pairs of variables, because the pairwise-complete subset differs from pair to pair (Section 8.6).

8.5 Defects found and corrected

Testing found several defects, all corrected. The row cache stored a Drizzle query instead of its result. Drizzle queries are lazy and run again on every `await`, so the "cached" selection was queried again for every view (1.5 s each); the loader now stores a settled promise. The server export imported a module that defined a browser-side context, which Next.js rejects in server code, so the dataset export failed; the context moved to the client component that uses it. The year-range chip shared with the Explore page displayed "2,015–2,025" (a Phase 1 defect). A chart's loading placeholder and the Predict page's layout at phone width were also fixed. The Python requirements had been pinned to versions that need Python 3.11 or later, while the README documents Python 3.9; they are now minimum versions that support both.

A final audit against the specification found five more. The emissions-program filter listed in §4.1 had never been built; it now matches a whole program code, because CAMPD stores lists such as "ARP, CSNOX, CSOSG2" and a substring match would let "CSNOX" match "CSNOXOS". The default 2015–2025 range applied only to a URL with no parameters at all, so a link to a tab (`/analyze?tab=charts`) analysed every year, including the partial 2026; it now applies whenever the URL sets no filter. The row cache could miss an import for up to five minutes; its key now includes the time of the latest import. A caption wrapped around a segmented control as an HTML `label` passed a click on the caption to the first button, which changed the choice; such captions are now plain groups. Sorting groups whose value was missing used an inconsistent comparison.

8.6 Known issues

- The Spearman correlation heat map takes about two seconds on the full selection. Ranking each variable once when it has no missing values would remove most of the cost.
- The state choropleth needs internet access to load the U.S. outlines from `cdn.plot.ly`.
- Histograms, box plots, and scatter plots of very large selections draw a sample, as the chart states. Statistics are always computed on every value.
- Fuel and unit type are stored once per unit (Section 10).

8.7 Assessment against the goals and objectives

Table 10 assesses the nine goals of the Phase 2 specification and the course objectives that Phase 2 addresses.

Table 10: Assessment of the Phase 2 goals and course objectives.

Goal or objective	Verdict	Evidence
G1 Descriptive statistics	Met	Distribution tab; checked against NumPy and SciPy.
G2 Comparisons	Met	Compare groups tab with tests and plain-language readings (Q2).
G3 Interactive visualization	Met	Seven chart types with export.
G4 TRACI impacts	Met	Four categories TRACI 2.2 defines for these pollutants (Q3).
G5 Trends, relationships, unusual observations	Met	Q1, the correlation outlier, far-out fences.
G6 Prediction models	Met	Two models beat the baseline on held-out years, with intervals; neither beats persistence, and Q4 says why.
G7 Sensitivity and scenarios	Met	With range warnings.
G8 Saving and exporting	Met	Every result exports with metadata.
G9 Explaining methods and limitations	Met	Definitions, captions, and caveats in the interface; this report.
O8 Environmental indicators	Met	Derived indicators and TRACI categories.
O9 Multi-indicator scoring	Not met	Not in the Phase 2 specification; not built.
O10 Interactive charts of results	Met	Every Analyze and Predict tab.
O11 Test and deploy	Met	81 + 10 automated tests; hosted at `https://cs396-project.vercel.app`.
O12 Document sources, assumptions, equations, units, limitations	Met	README, `DATABASE_BREAKDOWN.md`, this report.

9 Discussion

9.1 Effectiveness of techniques and tools

Treating every derived indicator and TRACI category as an ordinary variable, with one definition, unit, aggregation rule, and missing-value rule, was the most useful decision of the phase. Statistics, charts, comparisons, impacts, and exports are each written once and work for all 29 variables; adding a variable means adding its definition. Reusing the Phase 1 filters for the analytical dataset meant the dataset builder came with every filter, its URL encoding, and its tests already in place.

Offline training kept pandas and scikit-learn for fitting and tuning and TypeScript for interactive use. The cost is that preprocessing is written twice, in `train.py` and `model.ts`. The parity test makes this safe: it fails if one is changed without the other.

9.2 Technical choices and alternatives

Flask and pandas at request time. The specification recommends computing analyses with pandas behind Flask routes. Rewriting the Phase 1 application in Flask was not realistic, and calling Python on each request (as Phase 1 does for uploads) would add a process start to every chart and would not run on the hosted build. The statistics needed are few and well defined, so

implementing them in TypeScript and checking them against SciPy was the smaller and safer option.

Plotly.js. Plotly provides every chart type the specification lists, including box plots, heat maps, and state maps, with tooltips, zoom, and PNG export, so one component wraps it for all charts. Its bundle is large, so it is loaded only when a chart is first shown.

Ridge and a decision tree. The two models were chosen to behave differently. Ridge is smooth and gives each input one global effect, which makes sensitivity curves easy to read. The tree captures interactions and thresholds and is much more accurate, but its response to one input is a step function and it may ignore an input entirely. The scenario disagreement in Section 6 follows directly from this, and showing both models makes the uncertainty visible. In hindsight, the persistence reference should have been computed first: it sets the bar a model has to clear, and it showed that the inputs, not the choice between these two models, limit the accuracy.

9.3 Challenges and lessons learned

The TRACI workbook needed care: it lists both nitrogen oxides and nitrogen dioxide with different smog factors, and it has no eutrophication column, although our plan assumed one. Reading the workbook itself, not a summary of it, avoided both errors. The parity test taught a similar lesson: a TypeScript copy of a scikit-learn tree that looks correct still differs in rare cases unless it reproduces the library's 32-bit comparison. Finally, the correlation outlier showed that statistics over raw emissions data should be robust by default, since a handful of unit-years with almost no generation can produce a near-perfect Pearson correlation that describes nothing.

10 Limitations and future improvements

10.1 Current limitations

Potential, not actual, impacts. TRACI factors express potential contributions for comparison. They ignore stack height, local population, weather, and background concentrations, and they say nothing about actual illness or damage. Only the three pollutants CAMPD reports are covered; particulate matter, mercury, and other emissions are not, and TRACI 2.2 has no eutrophication factor for these substances.

Missing pollutants remove categories. A category is missing for a unit-year when one of its pollutants is not reported. About 13.7 % of 2015–2025 unit-years lack SO_2, so their acidification and particulate results are missing, and group totals cover only the unit-years that report every contributing pollutant. The Overview tab's data-quality section shows these counts.

Fuel and unit type are not stored per year. As in Phase 1, fuel and unit type are stored once per unit, so a unit that switched from coal to gas is treated as a gas unit in every year, both in grouped statistics and as a model input.

Statistical independence. Unit-years of the same unit are correlated, but the tests treat them as independent, which overstates significance. A mixed model, or tests on one value per unit, would be more rigorous.

Model responses are not causal. Both models learn associations in observational data. A scenario shows how a model responds when one input changes and everything else is fixed, which may be physically impossible (for example more gross load with the same heat input). The previous year's NO_x rate, the most useful history input, also encodes the unit's existing controls, which weakens the models' response to a changed control and makes them slow to follow a unit whose rate changes, as Ghent unit 2 shows. Range warnings check each input separately, not the combination.

The models do not beat persistence. Last year's NO_x rate times this year's heat input predicts the test years more accurately than either model (Q4), and three reformulations did not close the gap. The models are useful for exploring responses to inputs, not as the best forecast of a unit's emissions.

Other limitations. Analysis configurations are saved only as URLs. The year-over-year variables compare consecutive years only, so a unit with a missing year has no change for the following year.

10.2 Future improvements

Storing fuel and unit type per year, as Phase 1 already proposed, would correct both the grouped statistics and the model inputs. Adding the previous year's controls as inputs would let a model see a control being installed, the change persistence cannot anticipate. Tests clustered by unit would give honest p-values. Gradient-boosted trees with monotonic constraints, or separate models for coal and gas, would likely improve accuracy and give a more plausible response to controls. A joint out-of-distribution check (for example the distance to the nearest training unit-year) would make range warnings stricter. Adding the TRACI factors for particulate matter and mercury would become worthwhile once CAMPD's mercury data are imported.

11 Individual contributions

Phase 2 kept the roles of Phase 1, a back-end lead and a front-end lead, with testing, documentation, the report, and the presentation shared.

Raaj Banga (back-end lead). The analytical data layer and the models: the variable catalogue of derived indicators and year-over-year changes with their missing-value rules; the statistics and hypothesis tests and their checks against SciPy; the TRACI 2.2 extract, the `traci_factors` and `models` tables, and the impact calculation; the analysis and prediction procedures and the dataset export; the training pipeline in `ml/train.py`, the TypeScript inference module, and the parity and pipeline tests; and the README, `DATABASE_BREAKDOWN.md`, and this report.

Arnav Bawankule (front-end lead). The Phase 2 user interface: the Analyze page and its Overview, Distribution, Relationships, Compare groups, and TRACI impacts tabs; the Predict page and its Models and What-if tabs; the shared Plotly chart component, its theming, and the export controls that save each chart and table with its metadata; and the presentation slides.

12 References and acknowledgments

12.1 Data

Emissions, operating, facility, and unit data are those of the Phase 1 database, from the EPA Clean Air Markets Program Data (CAM API, `https://www.epa.gov/power-sector/cam-api-portal`), a U.S. government work in the public domain. TRACI 2.2 characterization factors are from the EPA workbook `traci_2_2.xlsx` [9], downloaded on 30 October 2026 (SHA-256 `80383dec…3725334`); only the six factors in Table 6 are redistributed, in `data/traci/`.

12.2 Reused code and assets

Phase 2 builds on the Phase 1 code base and its libraries. It adds Plotly.js [8] for charts, fflate for ZIP files, and, in the offline pipeline, pandas, NumPy, scikit-learn, joblib, SciPy, and pytest, all used under their open-source licences. No external example code was copied.

12.3 Generative AI disclosure

In line with the course policy, we disclose that generative AI was used throughout this phase. We used Anthropic's Claude, through the Claude Code tool, to help plan the phase, write and review application, pipeline, and test code, investigate defects, capture screenshots, and draft the README, the database documentation, and this report. The team defined the analytical questions, the prediction target, and the design decisions, reviewed all generated code and text, and accepted changes only after checking them with the type checker, linter, unit tests, reference values from NumPy and SciPy, and manual use of the application. Every number in this report was produced by running the application or the training pipeline against the committed database. The team is responsible for the content of the code and of this report.

Bibliography

[1] Jane Bare. TRACI 2.0: The tool for the reduction and assessment of chemical and other environmental impacts 2.0. *Clean Technologies and Environmental Policy*, 13(5):687–696, 2011.

[2] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, Belmont, CA, 1984.

[3] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[4] Rob J. Hyndman and Yanan Fan. Sample quantiles in statistical packages. *The American Statistician*, 50(4):361–365, 1996.

[5] William H. Kruskal and W. Allen Wallis. Use of ranks in one-criterion variance analysis. *Journal of the American Statistical Association*, 47(260):583–621, 1952.

[6] Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *Annals of Mathematical Statistics*, 18(1):50–60, 1947.

[7] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot,

and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[8] Plotly Technologies Inc. Plotly.js: Open-source JavaScript graphing library, 2025. Version 3.7. `https://plotly.com/javascript/`.

[9] U.S. Environmental Protection Agency. TRACI 2.2: Tool for the reduction and assessment of chemical and other environmental impacts, 2022. Characterization factor workbook `traci_2_2.xlsx`, updated 2 March 2022. `https://www.epa.gov/chemical-research/tool-reduction-and-assessment-chemicals-and-other-environmental-impacts-traci`, accessed 30 October 2026.

[10] Pauli Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[11] B. L. Welch. The generalization of "Student's" problem when several different population variances are involved. *Biometrika*, 34(1–2):28–35, 1947.

We used AI to help with this project.